

World Happiness Update Report: 2026

About Dataset

This dataset contains the full set of country-level rankings and happiness scores from the World Happiness Update Report 2026, released on March 20, 2026. It includes data for 147 countries spanning all major regions of the world.

Each entry corresponds to a single country and reports its overall happiness score along with six key explanatory factors: income (economic prosperity), social support, life expectancy (health), freedom to make life choices, generosity, and perceived corruption.

The data is drawn from the Gallup World Poll, which conducts nationally representative surveys in more than 140 countries annually. To ensure greater stability and comparability, the scores in this edition are averaged over the 2023–2025 period, minimizing short-term fluctuations.

This dataset is well-suited for a range of analytical tasks, including exploratory data analysis (EDA), correlation analysis, predictive modeling, and geospatial visualization. It can also be integrated with other datasets—such as GDP, population statistics, or digital behavior metrics—for more comprehensive research.

A key highlight of the 2026 report is its focus on social media and youth wellbeing. The findings suggest that young adults in English-speaking countries have seen notable declines in life satisfaction over the past two decades, with researchers pointing to high levels of social media use as a contributing factor.

```
import pandas as pd          # Data manipulation and analysis
import numpy as np           # Numerical calculations

import matplotlib.pyplot as plt # Data visualization
import seaborn as sns         # Statistical data visualization

from scipy import stats      # Statistical analysis

import warnings               # Handle warning messages
warnings.filterwarnings("ignore")

%pip install scipy

Collecting scipy
  Downloading scipy-1.18.1-cp313-cp313-macosx_14_0_arm64.whl.metadata
(62 kB)
Requirement already satisfied: numpy<2.8,>=2.0.0 in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/
site-packages (from scipy) (2.2.3)
Downloading scipy-1.18.1-cp313-cp313-macosx_14_0_arm64.whl (20.5 MB)
_____ 20.5/20.5 MB 6.5 MB/s
0:00:036.5 MB/s eta 0:00:01:01
```

```
Installing collected packages: scipy
Successfully installed scipy-1.18.1
```

```
[notice] A new release of pip is available: 25.2 -> 26.2.1
[notice] To update, run: pip install --upgrade pip
Note: you may need to restart the kernel to use updated packages.
```

```
import scipy
print(scipy.__version__)
```

```
1.18.1
```

```
df = pd.read_csv("world_happiness_Update_report_2026.csv")
```

Data Cleaning and Proprocessing

```
df.head(10)# Top 10 rowdf.
```

	rank	country	gdp_per_capita	social_support
0	1	Finland	1.892	1.581
1	2	Iceland	1.874	1.611
2	3	Denmark	1.887	1.562
3	4	Costa Rica	1.254	1.421
4	5	Sweden	1.878	1.501
5	6	Norway	1.959	1.553
6	7	Netherlands	1.877	1.488
7	8	Israel	1.803	1.521
8	9	Luxembourg	2.096	1.502
9	10	Switzerland	1.993	1.551

	freedom	generosity	corruption	score
0	0.670	0.186	0.498	7.764
1	0.662	0.269	0.512	7.701
2	0.665	0.211	0.495	7.688
3	0.631	0.178	0.312	7.439
4	0.658	0.224	0.481	7.401
5	0.661	0.210	0.491	7.392
6	0.642	0.267	0.411	7.319

```

7    0.599    0.144    0.311    7.234
8    0.649    0.181    0.432    7.228
9    0.647    0.271    0.430    7.201

```

```
df.tail(10) # Last 10 row
```

```

      rank  country  gdp_per_capita  social_support \
137   138     Togo         0.271         0.651
138   139    Guinea         0.311         0.641
139   140    Haiti         0.241         0.631
140   141    Yemen         0.291         0.611
141   142   Lebanon         0.491         0.691
142   143  Zimbabwe         0.341         0.621
143   144  Congo DR         0.191         0.601
144   145 South Sudan         0.251         0.581
145   146    Rwanda         0.391         0.701
146   147 Afghanistan         0.241         0.341

```

```

      healthy_life_expectancy  freedom  generosity  corruption  score
137                0.321      0.221      0.141      0.011  3.221
138                0.311      0.211      0.131      0.011  3.201
139                0.351      0.201      0.101      0.011  3.181
140                0.421      0.181      0.071      0.011  2.891
141                0.691      0.191      0.071      0.021  2.707
142                0.381      0.211      0.091      0.011  2.811
143                0.311      0.201      0.141      0.011  2.761
144                0.251      0.181      0.131      0.011  2.711
145                0.431      0.381      0.201      0.141  2.661
146                0.361      0.071      0.071      0.011  1.446

```

Size of Dataset

```
df.size # Size of Dataset
```

```
1323
```

All columns Name

```
df.columns # All columns Name
```

```

Index(['rank', 'country', 'gdp_per_capita', 'social_support',
      'healthy_life_expectancy', 'freedom', 'generosity',
      'corruption',
      'score'],
      dtype='object')

```

Data types

```
df.dtypes # Data types
```

```
rank          int64
country       object
gdp_per_capita float64
social_support float64
healthy_life_expectancy float64
freedom       float64
generosity    float64
corruption    float64
score         float64
dtype: object
```

Information

```
df.info() # Information
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 147 entries, 0 to 146
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  ---                                -
0   rank                                  147 non-null   int64
1   country                             147 non-null   object
2   gdp_per_capita                       147 non-null   float64
3   social_support                       147 non-null   float64
4   healthy_life_expectancy              147 non-null   float64
5   freedom                             147 non-null   float64
6   generosity                           147 non-null   float64
7   corruption                           147 non-null   float64
8   score                               147 non-null   float64
dtypes: float64(7), int64(1), object(1)
memory usage: 10.5+ KB
```

Check the number of missing (NaN) values in each column

```
df.isnull().sum()## Check the number of missing (NaN) values in each column
```

```
rank          0
country       0
gdp_per_capita 0
```

```

social_support      0
healthy_life_expectancy  0
freedom             0
generosity          0
corruption          0
score              0
dtype: int64

```

Count the total number of duplicate

```
df.duplicated().sum() # Count the total number of duplicate
```

```
np.int64(0)
```

```
df.describe()
```

	rank	gdp_per_capita	social_support
count	147.000000	147.000000	147.000000
mean	74.000000	1.043095	1.122279
std	42.579338	0.588589	0.291444
min	1.000000	0.171000	0.341000
25%	37.500000	0.471000	0.851000
50%	74.000000	1.021000	1.181000
75%	110.500000	1.596000	1.361000
max	147.000000	2.121000	1.611000

	freedom	generosity	corruption	score
count	147.000000	147.000000	147.000000	147.000000
mean	0.457673	0.162884	0.161340	5.205694
std	0.135096	0.054843	0.135871	1.393472
min	0.071000	0.061000	0.011000	1.446000
25%	0.371000	0.131000	0.051000	3.901000
50%	0.481000	0.161000	0.121000	5.431000
75%	0.561000	0.196000	0.246000	6.291000
max	0.671000	0.301000	0.512000	7.764000

```
# Find duplicate rows in the DataFrame
```

```
duplicate_rows = df[df.duplicated()] # df.duplicated() returns True
for duplicate rows (after the first occurrence)
```

```

# Print message for clarity
print("\nDuplicate Rows:")

# Display the duplicate rows
print(duplicate_rows)

Duplicate Rows:
Empty DataFrame
Columns: [rank, country, gdp_per_capita, social_support,
healthy_life_expectancy, freedom, generosity, corruption, score]
Index: []

# Drop duplicate rows based on all columns
df = df.drop_duplicates()

# Verify
print("After dropping duplicates, shape:", df.shape)

After dropping duplicates, shape: (147, 9)

# =====
# Standardize Text Casing and Whitespace
# =====

# Remove leading and trailing whitespace from country names
df["country"] = df["country"].str.strip()

# Standardize country name casing
df["country"] = df["country"].str.title()

# Check the country names after formatting
df["country"].head(10)

0      Finland
1      Iceland
2      Denmark
3    Costa Rica
4      Sweden
5      Norway
6  Netherlands
7      Israel
8    Luxembourg
9    Switzerland
Name: country, dtype: object

# Check for leading/trailing whitespace
print("Countries with extra whitespace:",
      (df["country"] != df["country"].str.strip()).sum())

# Check number of unique countries

```

```
print("Unique countries:", df["country"].nunique())

# Check dataset rows
print("Total rows:", len(df))

Countries with extra whitespace: 0
Unique countries: 130
Total rows: 147
```

1. Analyze country names

```
# =====
# Country Name Formatting Analysis
# =====

# Check data type
print("Country column data type:", df["country"].dtype)

# Check total number of country names
print("Total country names:", len(df["country"]))

# Check unique country names
print("Unique country names:", df["country"].nunique())

# Check missing country names
print("Missing country names:", df["country"].isnull().sum())

# Check duplicate country names
print("Duplicate country names:", df["country"].duplicated().sum())

Country column data type: object
Total country names: 147
Unique country names: 130
Missing country names: 0
Duplicate country names: 17
```

A. First, identify which countries are duplicated

```
# Find countries appearing more than once
duplicate_countries =
df[df["country"].duplicated(keep=False)].sort_values("country")

print("Countries with duplicate names:")
print(duplicate_countries[["rank", "country", "score"]])

Countries with duplicate names:
   rank  country  score
135  136  Burkina Faso  3.261
95    96  Burkina Faso  4.931
```

133	134	Chad	3.301
98	99	Chad	4.151
138	139	Guinea	3.201
102	103	Guinea	4.071
122	123	Haiti	3.521
139	140	Haiti	3.181
15	16	Kosovo	6.910
35	36	Kosovo	6.321
141	142	Lebanon	2.707
92	93	Lebanon	3.701
103	104	Liberia	4.051
136	137	Liberia	3.241
134	135	Malawi	3.281
97	98	Malawi	4.891
94	95	Mali	4.951
131	132	Mali	3.341
93	94	Mozambique	4.971
129	130	Mozambique	3.381
100	101	Niger	4.111
132	133	Niger	3.321
110	111	Rwanda	3.911
145	146	Rwanda	2.661
127	128	South Sudan	3.421
144	145	South Sudan	2.711
137	138	Togo	3.221
104	105	Togo	4.031
121	122	Yemen	3.541
140	141	Yemen	2.891
96	97	Zambia	4.911
130	131	Zambia	3.361
111	112	Zimbabwe	3.891
142	143	Zimbabwe	2.811

```
# Count occurrences of each country
country_counts = df["country"].value_counts()
```

```
print("Countries appearing more than once:")
print(country_counts[country_counts > 1])
```

Countries appearing more than once:

country	
Niger	2
Kosovo	2
Mali	2
Burkina Faso	2
Zambia	2
Malawi	2
Chad	2
Guinea	2
Liberia	2

```
Togo                2
Rwanda              2
Zimbabwe            2
Lebanon             2
Mozambique          2
Haiti               2
South Sudan         2
Yemen               2
Name: count, dtype: int64
```

```
# Check complete duplicate rows
print("Complete duplicate rows:", df.duplicated().sum())
```

```
Complete duplicate rows: 0
```

```
# Check whether ranks are duplicated
print("Duplicate ranks:", df["rank"].duplicated().sum())
```

```
# Display duplicated ranks if any
print(df[df["rank"].duplicated(keep=False)].sort_values("rank"))
```

```
Duplicate ranks: 0
Empty DataFrame
Columns: [rank, country, gdp_per_capita, social_support,
healthy_life_expectancy, freedom, generosity, corruption, score]
Index: []
```

```
# Check whether country and rank combinations are duplicated
print(
    "Duplicate country-rank combinations:",
    df.duplicated(subset=["country", "rank"]).sum()
)
```

```
Duplicate country-rank combinations: 0
```

```
duplicate_countries =
df[df["country"].duplicated(keep=False)].sort_values("country")
```

```
duplicate_countries
```

	rank	country	gdp_per_capita	social_support	\
135	136	Burkina Faso	0.271	0.671	
95	96	Burkina Faso	0.391	0.941	
133	134	Chad	0.271	0.691	
98	99	Chad	0.381	0.821	
138	139	Guinea	0.311	0.641	
102	103	Guinea	0.431	0.831	
122	123	Haiti	0.361	0.801	
139	140	Haiti	0.241	0.631	
15	16	Kosovo	1.021	1.298	
35	36	Kosovo	1.021	1.301	

141	142	Lebanon	0.491	0.691
92	93	Lebanon	0.681	0.941
103	104	Liberia	0.291	0.841
136	137	Liberia	0.181	0.661
134	135	Malawi	0.201	0.681
97	98	Malawi	0.291	0.951
94	95	Mali	0.471	0.951
131	132	Mali	0.341	0.711
93	94	Mozambique	0.371	0.961
129	130	Mozambique	0.261	0.721
100	101	Niger	0.251	0.811
132	133	Niger	0.181	0.701
110	111	Rwanda	0.541	0.921
145	146	Rwanda	0.391	0.701
127	128	South Sudan	0.361	0.691
144	145	South Sudan	0.251	0.581
137	138	Togo	0.271	0.651
104	105	Togo	0.391	0.821
121	122	Yemen	0.411	0.831
140	141	Yemen	0.291	0.611
96	97	Zambia	0.551	0.961
130	131	Zambia	0.411	0.731
111	112	Zimbabwe	0.491	0.881
142	143	Zimbabwe	0.341	0.621

	healthy_life_expectancy	freedom	generosity	corruption	score
135	0.341	0.241	0.141	0.011	3.261
95	0.451	0.421	0.181	0.061	4.931
133	0.331	0.251	0.151	0.011	3.301
98	0.371	0.331	0.171	0.051	4.151
138	0.311	0.211	0.131	0.011	3.201
102	0.451	0.361	0.171	0.041	4.071
122	0.571	0.261	0.131	0.021	3.521
139	0.351	0.201	0.101	0.011	3.181
15	0.782	0.571	0.212	0.251	6.910
35	0.791	0.561	0.181	0.221	6.321
141	0.691	0.191	0.071	0.021	2.707
92	0.791	0.271	0.091	0.051	3.701
103	0.441	0.391	0.221	0.051	4.051
136	0.321	0.231	0.171	0.011	3.241
134	0.371	0.261	0.231	0.011	3.281
97	0.481	0.441	0.261	0.061	4.891
94	0.471	0.431	0.191	0.071	4.951
131	0.391	0.281	0.161	0.011	3.341
93	0.501	0.441	0.211	0.081	4.971
129	0.381	0.291	0.181	0.011	3.381
100	0.371	0.371	0.181	0.051	4.111
132	0.361	0.271	0.171	0.011	3.321
110	0.581	0.521	0.261	0.191	3.911

145	0.431	0.381	0.201	0.141	2.661
127	0.341	0.221	0.171	0.011	3.421
144	0.251	0.181	0.131	0.011	2.711
137	0.321	0.221	0.141	0.011	3.221
104	0.451	0.371	0.181	0.041	4.031
121	0.571	0.241	0.091	0.031	3.541
140	0.421	0.181	0.071	0.011	2.891
96	0.521	0.451	0.201	0.071	4.911
130	0.421	0.301	0.191	0.021	3.361
111	0.521	0.381	0.131	0.051	3.891
142	0.381	0.211	0.091	0.011	2.811

2. Check leading/trailing whitespace

```
# Check for leading or trailing spaces
extra_spaces = (df["country"] != df["country"].str.strip()).sum()

print("Country names with extra whitespace:", extra_spaces)

Country names with extra whitespace: 0
```

3. Check uppercase/lowercase inconsistencies

```
# Display country names that are not in title case
not_title_case = df[df["country"] != df["country"].str.title()]

print("Country names with inconsistent capitalization:",
      len(not_title_case))

not_title_case[["country"]]

Country names with inconsistent capitalization: 0

Empty DataFrame
Columns: [country]
Index: []
```

4. Check special characters

```
# Check country names containing special characters
special_characters = df[
    df["country"].str.contains(r"^[a-zA-Z\s\-'\.]", regex=True,
na=False)
]
```

```
print("Country names containing special characters:",
      len(special_characters))

special_characters[["country"]]

Country names containing special characters: 0

Empty DataFrame
Columns: [country]
Index: []
```

5. Check country-name uniqueness

```
# Check whether different country names may represent duplicates
country_counts = df["country"].value_counts()

print("Countries appearing more than once:")
print(country_counts[country_counts > 1])

Countries appearing more than once:
country
Niger                2
Kosovo               2
Mali                 2
Burkina Faso         2
Zambia               2
Malawi               2
Chad                 2
Guinea               2
Liberia              2
Togo                 2
Rwanda               2
Zimbabwe             2
Lebanon              2
Mozambique           2
Haiti                2
South Sudan          2
Yemen                2
Name: count, dtype: int64
```

6. Check numerical columns

```
# =====
# Numerical Data Analysis
# =====

numeric_columns = [
```

```

    "gdp_per_capita",
    "social_support",
    "healthy_life_expectancy",
    "freedom",
    "generosity",
    "corruption",
    "score"
]

# Check data types
print(df[numeric_columns].dtypes)

# Check missing values
print("\nMissing values:")
print(df[numeric_columns].isnull().sum())

# Check infinite values
print("\nInfinite values:")
print(np.isinf(df[numeric_columns]).sum())

gdp_per_capita      float64
social_support      float64
healthy_life_expectancy float64
freedom             float64
generosity          float64
corruption          float64
score              float64
dtype: object

Missing values:
gdp_per_capita      0
social_support      0
healthy_life_expectancy 0
freedom             0
generosity          0
corruption          0
score              0
dtype: int64

Infinite values:
gdp_per_capita      0
social_support      0
healthy_life_expectancy 0
freedom             0
generosity          0
corruption          0
score              0
dtype: int64

```

7. Check numerical ranges

```
# Minimum and maximum values
range_check = df[numeric_columns].agg(["min", "max"]).T

print(range_check)
```

	min	max
gdp_per_capita	0.171	2.121
social_support	0.341	1.611
healthy_life_expectancy	0.251	0.961
freedom	0.071	0.671
generosity	0.061	0.301
corruption	0.011	0.512
score	1.446	7.764

8. Check for negative values

```
# Check negative values
negative_values = (df[numeric_columns] < 0).sum()

print("Negative values in numerical columns:")
print(negative_values)
```

```
Negative values in numerical columns:
gdp_per_capita      0
social_support      0
healthy_life_expectancy  0
freedom             0
generosity          0
corruption          0
score              0
dtype: int64
```

9. Final formatting validation

```
# =====
# Final Text Formatting Validation
# =====

print("Extra whitespace:",
      (df["country"] != df["country"].str.strip()).sum())

print("Unique countries:",
      df["country"].nunique())

print("Total rows:",
```

```

len(df))

print("Duplicate rows:",
      df.duplicated().sum())

Extra whitespace: 0
Unique countries: 130
Total rows: 147
Duplicate rows: 0
The history saving thread hit an unexpected error
(OperationalError('attempt to write a readonly database')).History
will not be written to the database.

# =====
# CONCLUSION
# =====

# The country names were checked for leading/trailing whitespace
# and capitalization inconsistencies.
#
# No unnecessary text transformations were applied.
#
# The dataset does not contain date columns, so date-format
# standardization is not applicable.
#
# No unit conversion was performed because there was no documented
# unit inconsistency requiring conversion.
#
# Numerical columns were checked for missing, negative, infinite,
# and potentially extreme values.
#
# Only necessary cleaning operations were performed to preserve
# the original information in the dataset.

```

Decide null-handling case by case (drop, impute, or flag) rather than applying one method everywhere

1. Check missing values

```

# =====
# NULL / MISSING VALUE ANALYSIS
# =====

# Check the number of missing values in each column
missing_values = df.isnull().sum()

```

```
print("Missing values in each column:")
print(missing_values)
```

```
Missing values in each column:
rank                0
country             0
gdp_per_capita      0
social_support      0
healthy_life_expectancy 0
freedom             0
generosity          0
corruption          0
score               0
dtype: int64
```

2. Check missing-value percentage

```
# Calculate missing-value percentage
missing_percentage = (df.isnull().sum() / len(df)) * 100

print("Missing-value percentage:")
print(missing_percentage)
```

```
Missing-value percentage:
rank                0.0
country             0.0
gdp_per_capita      0.0
social_support      0.0
healthy_life_expectancy 0.0
freedom             0.0
generosity          0.0
corruption          0.0
score               0.0
dtype: float64
```

3. Decide treatment case by case

```
# Conclusion:
# The dataset was checked for missing values on a column-by-column
basis.
# No missing values were found in any of the 9 columns.
# Therefore, no rows were dropped and no values were imputed.
# This avoids unnecessary modification of the original dataset.
```

Final Validation / Inventory Checks

```
# =====  
# FINAL DATA INVENTORY CHECK  
# =====  
  
# 1. Check dataset shape  
print("Dataset Shape:", df.shape)  
  
# 2. Check data types  
print("\nData Types:")  
print(df.dtypes)  
  
# 3. Check missing values  
print("\nMissing Values:")  
print(df.isnull().sum())  
  
# 4. Check total missing values  
print("\nTotal Missing Values:", df.isnull().sum().sum())  
  
# 5. Check duplicate rows  
print("\nDuplicate Rows:", df.duplicated().sum())  
  
# 6. Check number of unique countries  
print("\nUnique Countries:", df["country"].nunique())  
  
# 7. Check duplicate country names  
print("Duplicate Country Names:",  
      df["country"].duplicated().sum())  
  
# 8. Check numerical columns  
print("\nNumerical Summary:")  
print(df.describe())
```

Dataset Shape: (147, 9)

Data Types:

rank	int64
country	object
gdp_per_capita	float64
social_support	float64
healthy_life_expectancy	float64
freedom	float64
generosity	float64
corruption	float64
score	float64
dtype:	object

Missing Values:

rank	0
------	---

```
country          0
gdp_per_capita   0
social_support    0
healthy_life_expectancy 0
freedom          0
generosity       0
corruption       0
score            0
dtype: int64
```

Total Missing Values: 0

Duplicate Rows: 0

Unique Countries: 130

Duplicate Country Names: 17

Numerical Summary:

	rank	gdp_per_capita	social_support
healthy_life_expectancy \			
count	147.000000	147.000000	147.000000
147.000000			
mean	74.000000	1.043095	1.122279
0.694075			
std	42.579338	0.588589	0.291444
0.212594			
min	1.000000	0.171000	0.341000
0.251000			
25%	37.500000	0.471000	0.851000
0.491000			
50%	74.000000	1.021000	1.181000
0.771000			
75%	110.500000	1.596000	1.361000
0.881000			
max	147.000000	2.121000	1.611000
0.961000			

	freedom	generosity	corruption	score
count	147.000000	147.000000	147.000000	147.000000
mean	0.457673	0.162884	0.161340	5.205694
std	0.135096	0.054843	0.135871	1.393472
min	0.071000	0.061000	0.011000	1.446000
25%	0.371000	0.131000	0.051000	3.901000
50%	0.481000	0.161000	0.121000	5.431000
75%	0.561000	0.196000	0.246000	6.291000
max	0.671000	0.301000	0.512000	7.764000

Check country formatting again

```
# =====  
# FINAL COUNTRY FORMATTING CHECK  
# =====  
  
# Check leading/trailing whitespace  
print(  
    "Countries with extra whitespace:",  
    (df["country"] != df["country"].str.strip()).sum()  
)  
  
# Check empty country names  
print(  
    "Empty country names:",  
    (df["country"].str.strip() == "").sum()  
)  
  
# Check missing country names  
print(  
    "Missing country names:",  
    df["country"].isnull().sum()  
)  
  
Countries with extra whitespace: 0  
Empty country names: 0  
Missing country names: 0
```

Final quality summary

```
# =====  
# FINAL DATA QUALITY SUMMARY  
# =====  
  
print("===== FINAL DATA QUALITY CHECK =====")  
  
print("Rows:", len(df))  
print("Columns:", len(df.columns))  
print("Missing Values:", df.isnull().sum().sum())  
print("Duplicate Rows:", df.duplicated().sum())  
print("Unique Countries:", df["country"].nunique())  
print(  
    "Extra Whitespace:",  
    (df["country"] != df["country"].str.strip()).sum()  
)  
  
print("=====")
```

```
===== FINAL DATA QUALITY CHECK =====  
Rows: 147  
Columns: 9  
Missing Values: 0  
Duplicate Rows: 0  
Unique Countries: 130  
Extra Whitespace: 0  
=====
```

Final conclusion comment

```
# =====  
# FINAL VALIDATION CONCLUSION  
# =====  
  
# The inventory checks were re-run after the data-cleaning process.  
# The dataset contains 147 rows and 9 columns.  
# No missing values were found.  
# No complete duplicate rows were found.  
# Country names were checked for formatting and whitespace issues.  
# Numerical columns were re-checked for valid values and ranges.  
#  
# The final validation confirms that the cleaning process did not  
# introduce new data-quality issues and that the dataset is ready  
# for further analysis.  
# =====
```

□ Interview Questions — Data Cleaning & Preprocessing

1. How do you decide whether to drop or impute a missing value?

Answer:

I decide based on the **amount of missing data, importance of the column, and reason for the missing value.**

- If only a few rows have missing values and they are not important, I may **drop those rows.**
- If the column is important and has a small number of missing values, I may **impute** them using mean, median, or mode.
- For numerical data with outliers, **median** is often safer than mean.

- If missingness itself is meaningful, I may **flag it** instead of replacing it.
- If a column has too many missing values and provides little analytical value, I may consider **dropping the column**.

Example:

For my World Happiness dataset, all columns had **0 missing values**, so no dropping or imputation was required.

2. What's the difference between a duplicate row and a duplicate key?

Answer:

A **duplicate row** means the entire record is repeated with the same values across all columns.

A **duplicate key** means the value that should uniquely identify a record appears more than once, even if the other column values are different.

Example:

```
```text Country Rank Score India 100 5.5 India 100 5.5
```

## 3. How would you handle outliers differently from missing values?

### Answer:

Missing values and outliers are different data-quality problems, so I would not treat them the same way.

- **Missing value:** A value is absent. I may **drop, impute, or flag** it depending on the situation.
- **Outlier:** A value is present but unusually high or low compared with the rest of the data.

For outliers, I would first investigate whether they are:

- A genuine observation
- A data-entry error
- A measurement problem

I can use methods such as **IQR, Z-score, box plots, or domain-specific limits**.

I would **not automatically remove an outlier**, because it may represent an important real-world observation.

---

#### 4. How would you validate that a dataset is "clean" before starting analysis?

Answer:

I would perform a final data-quality checklist before analysis:

1. Check the **number of rows and columns**.
2. Check **missing values**.
3. Check **duplicate rows**.
4. Check **duplicate keys/identifiers**.
5. Verify **data types**.
6. Standardize **text formatting and whitespace**.
7. Check **numerical ranges and invalid values**.
8. Investigate **outliers**.
9. Verify **date formats and units**, if applicable.
10. Re-run all inventory checks after cleaning.
11. Review a sample of the cleaned data.
12. Export and save the final cleaned dataset.

For my World Happiness dataset, the final validation showed that there were **147 rows, 9 columns, no missing values, and no complete duplicate rows**. Therefore, the dataset was ready for further analysis.